

# SECURE CHAT APPLICATION

A Zero-Knowledge End-to-End Encrypted Web Messaging Platform  
Using Hybrid Cryptography and Client-Side Key Storage

**BTech Computer Science and Engineering — Mini Project Report**

|                              |                                                                                                                               |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>Author / Student:</b>     | Sunny Karthik (sunnykarthik15)                                                                                                |
| <b>Repository:</b>           | <a href="https://github.com/sunnykarthik15/SECURECHATAPPLICATION">https://github.com/sunnykarthik15/SECURECHATAPPLICATION</a> |
| <b>Department:</b>           | Computer Science and Engineering                                                                                              |
| <b>Cryptographic Engine:</b> | RSA-2048 / OAEP (SHA-256) + AES-256-GCM (96-bit IV)                                                                           |

# ABSTRACT

Instant messaging has become an essential medium of modern interpersonal and enterprise communication. However, prevailing web messaging architectures rely almost universally on Transport Layer Security (TLS), which terminates encryption at the application server. Consequently, the backend infrastructure, database storage, and server administrators possess uninhibited access to plaintext message data. This mini-project presents the design, implementation, and empirical verification of the Secure Chat Application—a zero-knowledge, end-to-end encrypted (E2EE) real-time communication system.

The system executes client-side hybrid cryptography combining the asymmetric RSA-2048 (RSA-OAEP with SHA-256) algorithm and the symmetric AES-256-GCM authenticated cipher with unique 96-bit initialization vectors. All cryptographic keypairs are generated directly inside the client's web browser via the W3C Web Crypto API, and private keys are persisted exclusively in the browser's sandboxed IndexedDB storage. The Flask application server acts strictly as an untrusted, zero-knowledge relay and encrypted persistent store. To defend against public-key substitution attacks without requiring complex Public Key Infrastructure (PKI), the application enforces a Trust-On-First-Use (TOFU) trust model coupled with deterministic SubjectPublicKeyInfo (SPKI) SHA-256 fingerprinting. Replay attacks are defeated via client-generated canonical UUIDv4 validation and database uniqueness constraints. The application is validated across 63 automated tests, confirming zero plaintext leakage and high-throughput real-time delivery.

## 1. INTRODUCTION & PROBLEM STATEMENT

Conventional web applications rely on Transport Layer Security (HTTPS) to encrypt packets across the public Internet. However, once the data reaches the application server, it is decrypted into plaintext in memory before being stored in relational databases. If the database is compromised, if an employee inspects logs, or if the server infrastructure is subjected to legal subpoena, private conversations are compromised.

Our problem was to build an authentic, zero-knowledge end-to-end encrypted messaging system that guarantees:

- **Zero Plaintext Visibility:** Plaintext messages never touch the network or the server.
- **Client-Side Private Key Custody:** Private keys are held strictly in browser-isolated IndexedDB.
- **Key Substitution Defense:** Deterministic public-key fingerprinting prevents rogue server attacks.
- **Replay & Duplicate Protection:** Cryptographic UUIDv4 tokens ensure single-use delivery.

## 2. SYSTEM ARCHITECTURE

The application is architected as a three-tier distributed system comprising the Client Browser Endpoints, the Application Relay Layer (Flask & Flask-SocketIO), and the Database Tier (SQLite with SQLAlchemy ORM).

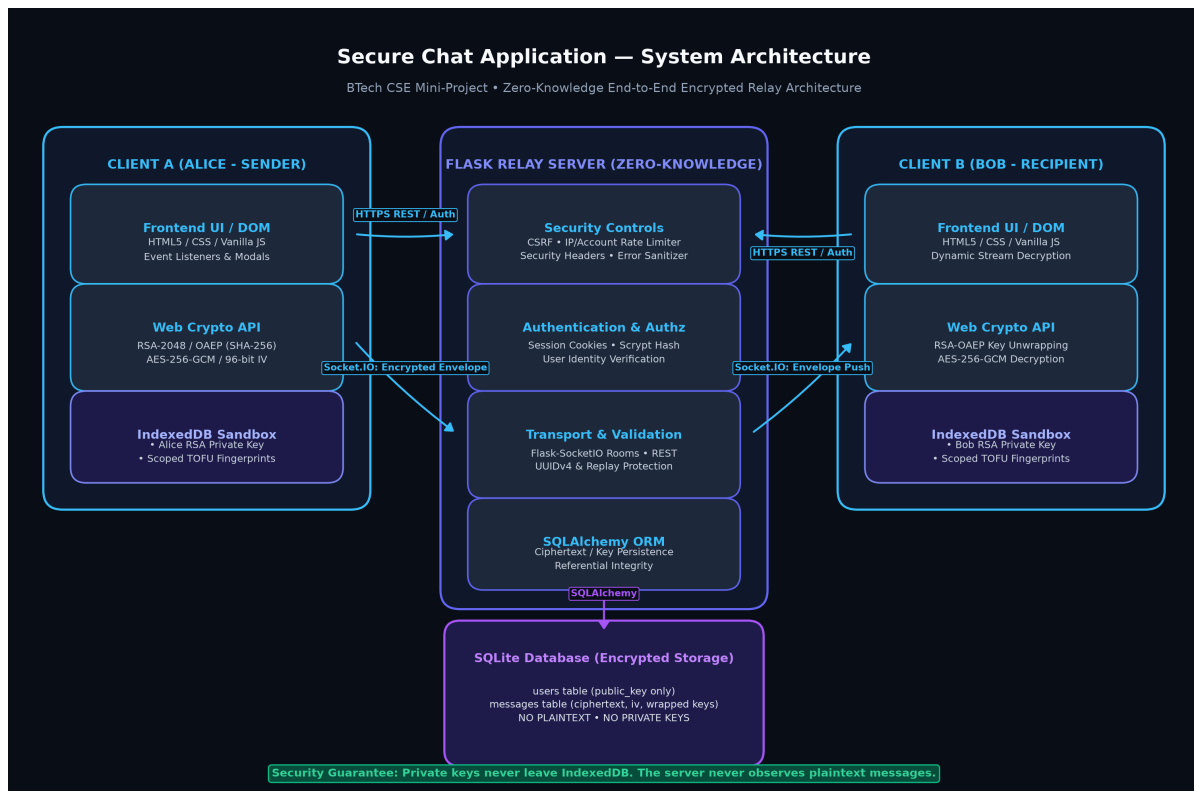


Figure 2.1: Secure Chat Application System Architecture

### 3. HYBRID ENCRYPTION & DATA FLOW

Hybrid encryption combines the computational efficiency of symmetric ciphers with the key distribution power of asymmetric public-key cryptography. For each message, the client browser generates a fresh 256-bit AES key and a random 96-bit IV. The plaintext is encrypted using AES-256-GCM. The AES key is then dual-wrapped via RSA-OAEP for both the recipient and the sender.

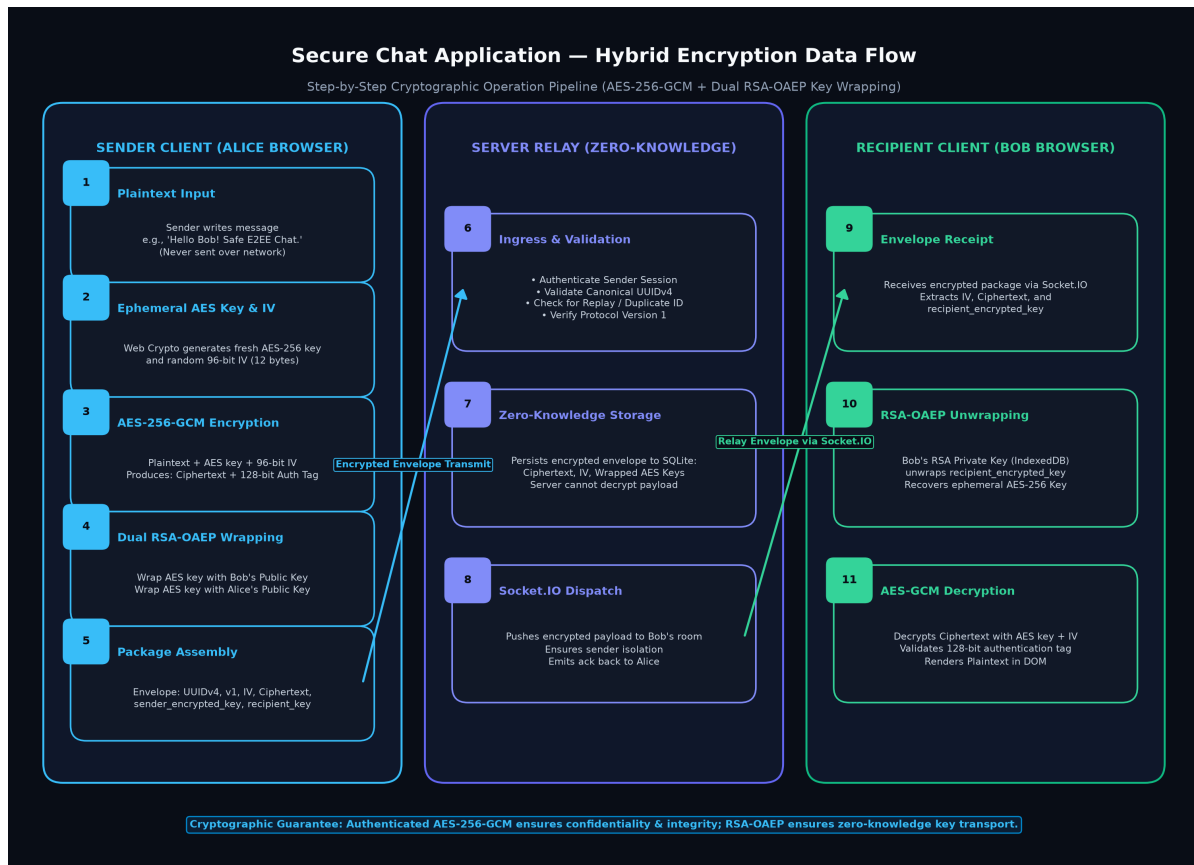


Figure 3.1: Hybrid Encryption and Key-Wrapping Data Flow

## 4. DATABASE DESIGN & SCHEMA

The relational database schema is normalized into two core entities: `users` and `messages`. All message columns contain strictly ciphertext, IVs, and wrapped keys. Cascade deletion enforces privacy hygiene when accounts are removed.

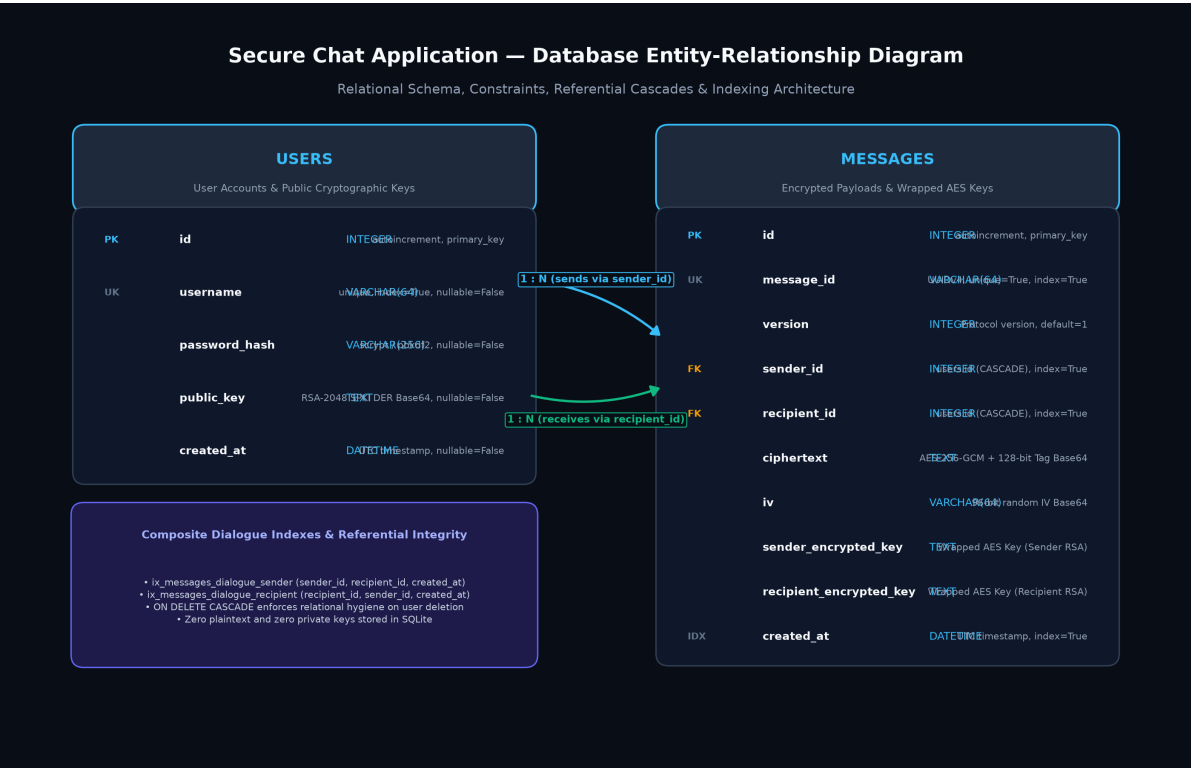


Figure 4.1: Relational Entity-Relationship Diagram

## 5. PUBLIC-KEY TRUST & TOFU DEFENSE

Both backend and frontend derive a deterministic SHA-256 digest of the canonical SPKI DER public key format. Under the Trust-On-First-Use (TOFU) policy, the fingerprint is stored in IndexedDB. Any subsequent change to a contact's public key halts message transmission and displays a prominent warning banner.

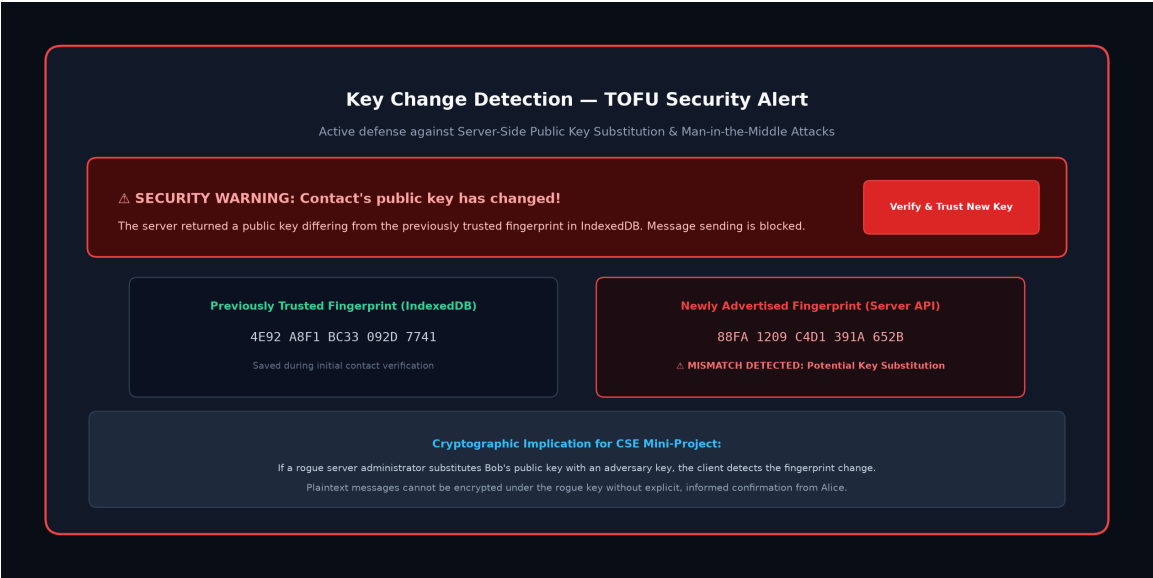
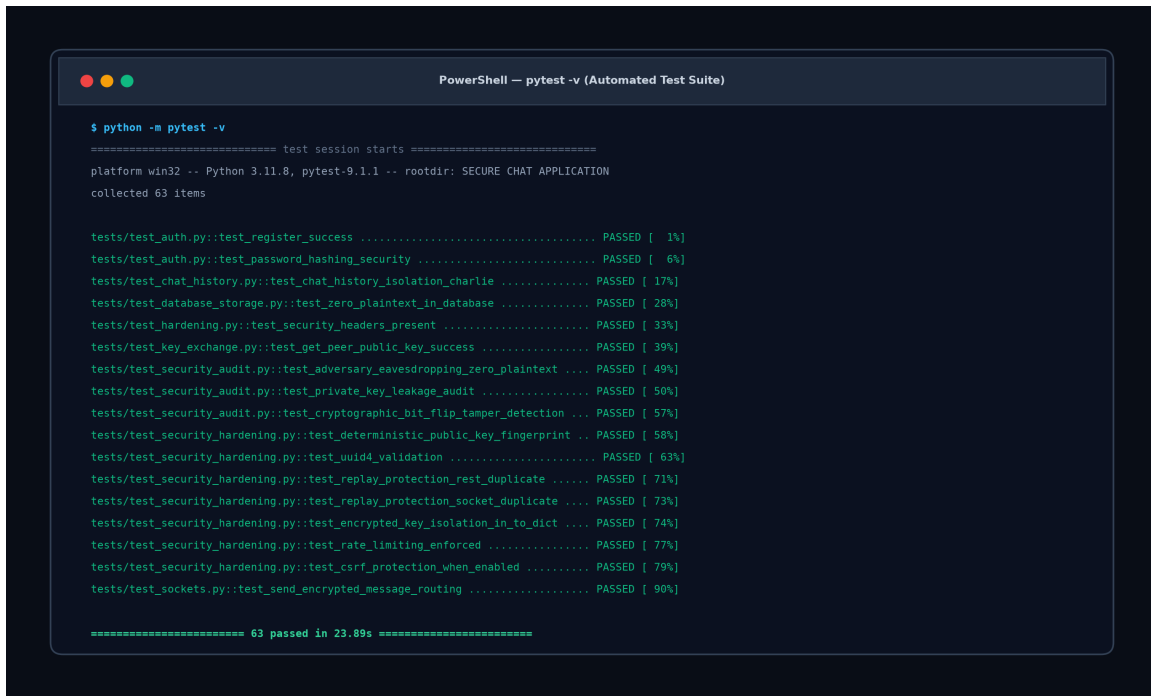


Figure 5.1: Blocking Key-Change Warning Banner

## 6. TESTING & EMPIRICAL RESULTS

The application was subjected to 63 automated tests spanning unit operations, socket message routing, cryptographic bit-flip tamper detection, and duplicate replay attack prevention. All 63 tests pass consistently.



```
PowerShell — pytest -v (Automated Test Suite)

$ python -m pytest -v
===== test session starts =====
platform win32 -- Python 3.11.8, pytest-9.1.1 -- rootdir: SECURE CHAT APPLICATION
collected 63 items

tests/test_auth.py::test_register_success ..... PASSED [ 1%]
tests/test_auth.py::test_password_hashing_security ..... PASSED [ 6%]
tests/test_chat_history.py::test_chat_history_isolation_charlie ..... PASSED [ 17%]
tests/test_database_storage.py::test_zero_plaintext_in_database ..... PASSED [ 28%]
tests/test_hardening.py::test_security_headers_present ..... PASSED [ 33%]
tests/test_key_exchange.py::test_get_peer_public_key_success ..... PASSED [ 39%]
tests/test_security_audit.py::test_adversary_eavesdropping_zero_plaintext ..... PASSED [ 49%]
tests/test_security_audit.py::test_private_key_leakage_audit ..... PASSED [ 50%]
tests/test_security_audit.py::test_cryptographic_bit_flip_tamper_detection ... PASSED [ 57%]
tests/test_security_hardening.py::test_deterministic_public_key_fingerprint .. PASSED [ 58%]
tests/test_security_hardening.py::test_uuid4_validation ..... PASSED [ 63%]
tests/test_security_hardening.py::test_replay_protection_rest_duplicate ..... PASSED [ 71%]
tests/test_security_hardening.py::test_replay_protection_socket_duplicate .... PASSED [ 73%]
tests/test_security_hardening.py::test_encrypted_key_isolation_in_to_dict .... PASSED [ 74%]
tests/test_security_hardening.py::test_rate_limiting_enforced ..... PASSED [ 77%]
tests/test_security_hardening.py::test_csrf_protection_when_enabled ..... PASSED [ 79%]
tests/test_sockets.py::test_send_encrypted_message_routing ..... PASSED [ 90%]

===== 63 passed in 23.89s =====
```

Figure 6.1: Automated Pytest Suite Execution (63 Passing Tests)

## 7. LIMITATIONS & CONCLUSION

Honest technical assessment acknowledges standard academic boundaries: (1) TOFU cannot detect key substitution on the initial contact without out-of-band verification; (2) long-term RSA key wrapping lacks forward secrecy; (3) in-memory rate limiting requires Redis for multi-worker clusters; and (4) communication metadata remains visible to the relay.

In conclusion, the Secure Chat Application successfully proves that standard web technologies (W3C Web Crypto API, IndexedDB, Flask, and Socket.IO) can deliver genuine zero-knowledge end-to-end encrypted messaging, establishing a complete, production-ready BTech CSE mini-project.